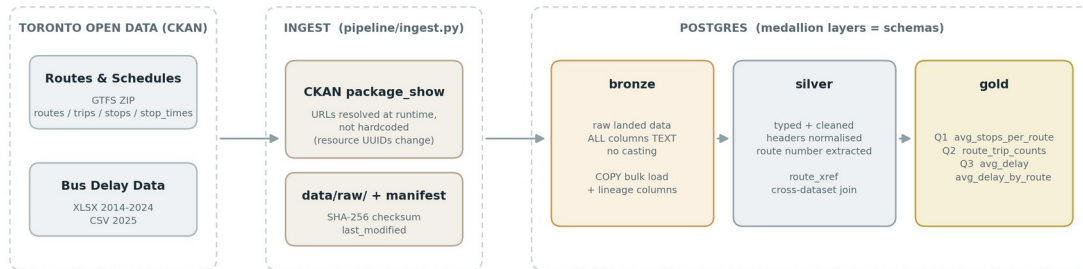


TTC Data Pipeline: Design Write-up

Yang Zhenzhao (Robert)

A pipeline over two City of Toronto open datasets: **TTC Routes & Schedules** (a GTFS feed) and **TTC Bus Delay Data**. Built with Postgres and Python in a medallion (bronze, silver, gold) architecture. It is ELT rather than ETL: Python orchestrates, and the transformations live in SQL files executed inside Postgres.



ELT: Python orchestrates; transformations run as SQL inside Postgres. Each stage is independently runnable and idempotent.

Each stage can also be run on its own, so a layer can be inspected before the next one is built. See the README for setup and run instructions.

Design choices

Docker. Mainly for the reviewer. One command brings up Postgres and runs the pipeline, with no local database install or environment setup. It also pins the Postgres version, so the result is reproducible rather than dependent on whatever happens to be on the machine.

Postgres. The data is inherently relational and the analyses are join heavy (routes to trips to stop_times, plus reconciling routes across the two datasets), so a relational engine with SQL is the natural fit. COPY bulk loads efficiently, which matters because stop_times is roughly 4.2M rows; row by row inserts would not be viable. It is also free and ubiquitous, so a reviewer can run it easily. I had not used Postgres before and wanted to learn it.

Medallion (bronze, silver, gold). Implemented as three Postgres schemas. It splits the work into stages that are separately inspectable and debuggable: raw landed data, cleaned and typed data, and analysis results. Because silver and gold are deterministic rebuilds from the layer below, a transform can be iterated on without re-downloading or re-loading anything.

Ingestion via the CKAN API, not hardcoded links. Download URLs contain resource UUIDs that change whenever the TTC republishes, so a hardcoded link silently rots. URLs are resolved at runtime from package_show. This was not theoretical: the GTFS feed changed mid-project, and the pipeline picked up the new file with no code change.

The three questions, and the assumptions behind them

Q1. Average number of stops per route. For each route I count the distinct stops it serves across all of its trips, then average that across routes. *Assumption: the trips present in the GTFS snapshot collectively cover every stop a route serves, so the union of stops over its trips is the route's true stop set.*

Q2. Route with the most frequent schedule updates. A single GTFS snapshot contains no update history, so this question has no literal answer from the data available. I interpret it as the most actively scheduled route: the route with the most trips. gold.route_trip_counts holds the full ranking, and the top row is the answer. *Assumption: trip counts are comparable across routes because the feed covers one fixed service period, so every route is counted over the same window rather than being sampled over differing durations.*

Q3. Average delay across all routes. Two caveats. First, the source is historical delay logs, not real-time updates. The dataset contains no real-time feed, so I have answered the question the available data can actually support. Second, "across all routes" is ambiguous: it could mean the grand mean over all delay records (each incident weighted equally), or the mean of per-route averages (each route weighted equally). gold.avg_delay reports the grand mean, and gold.avg_delay_by_route gives the per-route breakdown from which the route-weighted figure can be derived. *Assumption: I average Min Delay over records carrying a numeric value, including zero-minute records, since those are logged events with no measured delay.*

Data quality findings

The brief describes the sources as CSV. In practice, routes is a GTFS ZIP of linked tables, and delay is XLSX (2014 to 2024) plus CSV (2025). Handling this was most of the real work.

- **Header drift across years.** Delay vs Min Delay, Date vs Report Date, stray whitespace. A header audit surfaced these, and the loader normalises them onto a canonical column set.
- **The 2025 file renames columns outright:** Route to Line, Location to Station, Incident to Code. Mapped explicitly.
- **Route values are inconsistent.** Older years store 89, while 2025 stores 102 MARKHAM ROAD. Silver extracts the leading route number so every year joins consistently.
- **GTFS times can exceed 24 hours** (e.g. 25:30:00 for after-midnight trips), which is not a valid SQL time. Those columns are deliberately excluded from silver rather than forced into a cast that would fail.
- **Bronze is all TEXT** precisely because of the above: a malformed value cannot break ingestion. All typing and validation happens at the bronze to silver boundary.

Idempotency

Running the pipeline twice produces identical results.

- Schema DDL is CREATE ... IF NOT EXISTS and runs on every invocation, so the pipeline owns its own schema and works against any Postgres instance, not only one created by a first-boot init hook.
- Bronze GTFS is truncate and reload. Bronze delay is delete year then reload (_year is the partition key).
- Silver and gold are DROP + CREATE TABLE AS rebuilds from the layer below.
- Each stage runs inside a transaction, so a mid-load failure rolls back rather than leaving a half-loaded table.

Code guide

File	Responsibility
run.py	CLI entry point. Orchestrates the stages in order (ddl, ingest, bronze, silver, gold) and supports --stage, --reset and --local.
config.py	Central configuration. Reads .env, builds the connection string, holds the CKAN endpoints, paths, and load scope (FULL_LOAD vs the 2024 sample).
pipeline/db.py	Database helpers: open a connection, and run a .sql file in a single transaction (commits or rolls back as a unit).
pipeline/ingest.py	Resolves current download URLs from the CKAN API, streams files to data/raw/ with SHA-256 checksums, unzips GTFS, and writes manifest.json.
pipeline/bronze.py	Loads the raw files into the all TEXT bronze tables via COPY. Holds the delay header normalisation map that absorbs the cross-year drift.
pipeline/silver.py	Runs the silver SQL files.
pipeline/gold.py	Runs the gold SQL files.
sql/ddl/	00_schemas.sql (the three schemas) and 10_bronze.sql (bronze tables and ingestion_log). Idempotent.
sql/silver/	20_gtfs.sql (type and clean GTFS), 21_delay.sql (type and clean delay, extract route number), 22_route_xref.sql (cross-dataset reconciliation).
sql/gold/	30_q1_avg_stops.sql, 31_q2_schedule_updates.sql and 32_q3_avg_delay.sql, one file per question.
analysis/sample_queries.sql	Read-only route analysis queries against the gold tables.
docker-compose.yml	Postgres service (named volume, healthcheck) plus the pipeline service, which waits for the DB to pass its healthcheck.
Dockerfile	Image for the pipeline service.